西南交通大学
Southwest Jiaotong University

并发性 Concurrency: Mutual Exclusion and Synchronization

互斥

同步

Xiangyi Chen

xiangyichen@swjtu.edu.cn

mutural exclusion Synchronization

Outline

- Principles of Concurrency
- Software Solution with Busy Waiting
- Hardware Support with Machine Instruction
- Software Solution by OS and Language Complier
  - Semaphores 信号量 semaphores.
  - Monitors 管程
  - Message Passing 消息传递

Multiple Processes/Threads

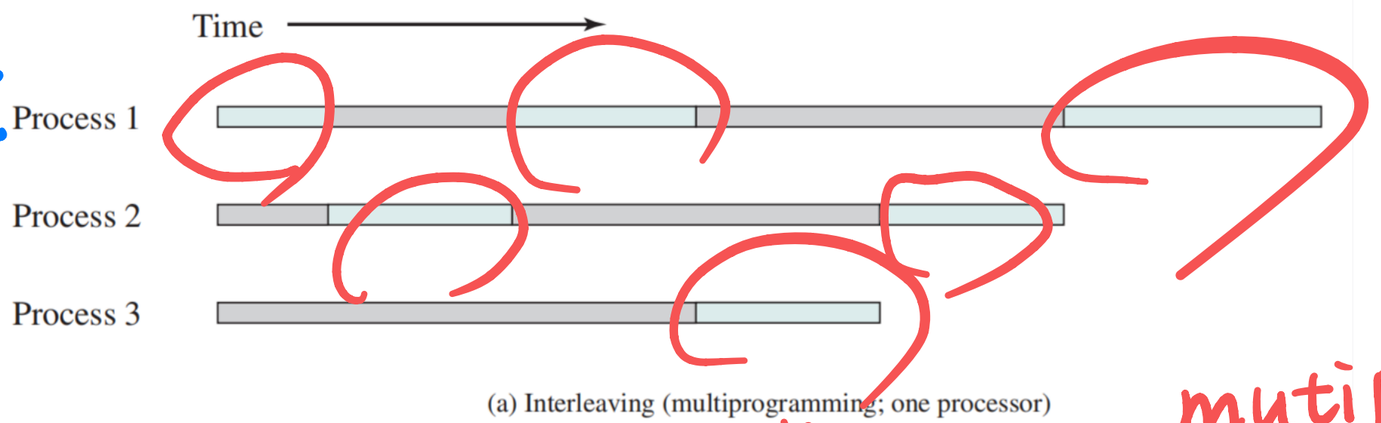
- **Operating System design is concerned with the management of processes and threads**
  - Multiprogramming——interleaving in single processor
  - Multiprocessing——interleaving and overlapping in multiple processor

Processes Concurrency

- **In the case of multiprogramming with uniprocessor, the relative speed of execution of processes cannot be predicted, depending on:**
  - activities of other processes
  - the way the OS handles interrupts
  - scheduling policies of the OS

Overleaving and Overlapping

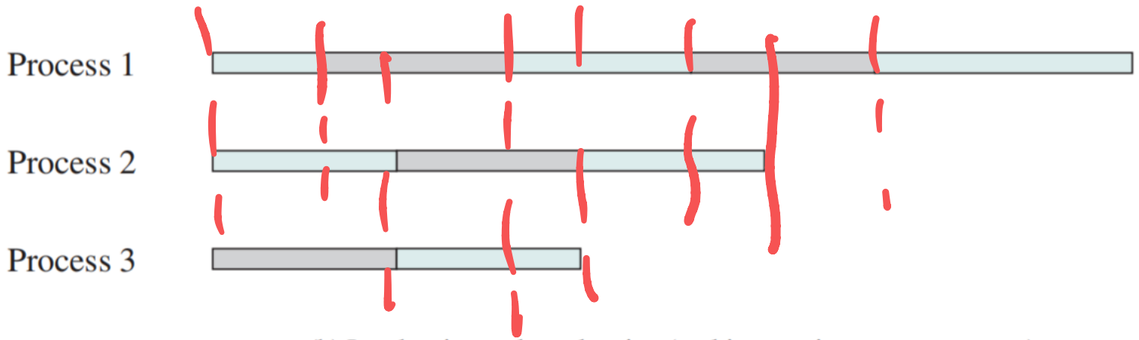
交错与重叠



(a) Interleaving (multiprogramming; one processor)

多道

mutiprogramming



(b) Interleaving and overlapping (multiprocessing; two processors)

mutiprucessing

Blocked Running

A Simple Example


```
int x = 0;
```

Shared between A and B

```
while(x < 10){
    x = x + 1;
}
fprintf("A wins");
```

A

```
while(x > -10){
    x = x - 1;
}
fprintf("B wins");
```

B

Two threads referencing x

1. Which one wins?
2. Is it guaranteed?

NON-DETERMINISTIC BEHAVIOUR 不确定表示

A programming example

Context switch

上下文切换

Let X=1			
T0	Process A	X=X+1	X=2
T1	Process A	X=X+1	X=3
T2	Process B	X=X-1	X=2
T3	Process B	X=X-1	X=1
T4	Process B	X=X-1	X=0
T5	Process A	X=X+1	X=1
T6	Process B	X=X-1	X=0

Concurrency 并发性

西南交通大学 Southwest Jiaotong University

Difficulties of Concurrency

- Sharing of global resources 共享
- Difficult for the OS to manage the allocation of resources optimally 分配
- Difficult to locate programming errors as results are not deterministic and reproducible

难以定位 不确定性 无法复现

Design issues in concurrency:

- communication among processes
- sharing/competing for resources
- Synchronization 同步
- allocation of processor

synchronization.

Key Terms Related to Concurrency

西南交通大学 Southwest Jiaotong University

核心术语

Atomic operation 原子操作	A function or action implemented as a sequence of one or more instructions that appears to be indivisible that is, no other process can see an intermediate state or interrupt the operation. The sequence of instruction is guaranteed to execute as a group, or not execute at all, having no visible effect on system state. Atomicity guarantees isolation from concurrent processes.
Critical section 临界区	A section of code within a process that requires access to shared resources, and that must not be executed while another process is in a corresponding section of code.
Deadlock 死锁	A situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something.
Livelock 活锁	A situation in which two or more processes continuously change their states in response to changes in the other process(es) without doing any useful work.
Mutual exclusion 互斥	The requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources.
Race condition 竞争条件	A situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution.
Starvation 饥饿	A situation in which a runnable process is overlooked indefinitely by the scheduler; although it is able to proceed, it is never chosen.

他们执行时序情况

忽略

Race Condition(竞争条件)

西南交通大学 Southwest Jiaotong University

- Occurs when multiple processes or threads read and write data items
- The final result depends on the order of execution
  - the "loser" of the race is the process that updates last and will determine the final value of the variable

```
Shared a;
P1: a=1;
P2: a=2;

If P1 first,  a=2; loser: P2;
If P2 first, a=1; loser: P1;
```

```
Shared b=1, c=2;
P1: b=b+c;
P2: c=b+c;

If P1 first,  b=3,c=5;
If P2 first, b=4, c=3;
```

Operating System Concerns

西南交通大学 Southwest Jiaotong University

- **What design and management issues are raised by the existence of concurrency:**
  - keep track of various processes (PCB)
  - Allocate and de-allocate resources for each active process:
    - Processor
    - Memory
    - Files
    - I/O devices
  - Protect the data and physical resources of each process **against interference** by other processes
  - Ensure that the **functioning and outputs are independent of the processing speed** at which its execution is carried out relative to the speed of other concurrent processes.

This is subject of this chapter

Process Interaction 进程交互

西南交通大学 Southwest Jiaotong University

Degree of Awareness	Relationship	Influence that One Process Has on the Other	Potential Control Problems
Processes unaware of each other	Competition 竞争	• Results of one process independent of the action of others 独立 • Timing of process may be affected 时序	• Mutual exclusion 互斥 • Deadlock (renewable resource) 死锁 • Starvation 饥饿
Processes indirectly aware of each other (e.g., shared object) 协作共享	Cooperation by sharing	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Mutual exclusion • Deadlock (renewable resource) • Starvation • Data coherence 数据一致性
Processes directly aware of each other (have communication primitives available to them) 通信协作	Cooperation by communication	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Deadlock (consumable resource) 消耗性 • Starvation

可复用

Requirements for Mutual Exclusion

- Only one process at a time is allowed into its critical section
- A process that halts in its noncritical section must do so without interfering with other processes.
- No deadlock or starvation
- When no process is in a critical section, any process that requests entry to its critical section must be permitted to enter without delay
- No assumptions are made about relative process speeds or number of processes
- A process remains inside its critical section for a finite time

Illustration of Mutual Exclusion

示例

```
/* PROCESS 1 * void  P1
{
  while  (true) {
   /* preceding code */;
   entercritical (Ra);
   /* critical section */;
   exitcritical (Ra);
   /* following code */;
  }
}
```

```
/* PROCESS 2 * void  P2
{
  while  (true) {
   /* preceding code */;
   entercritical (Ra);
   /* critical section */;
   exitcritical (Ra);
   /* following code */;
  }
}
```

• • •

```
/* PROCESS n * void  Pn
{
  while  (true) {
   /* preceding code */;
   entercritical (Ra);
   /* critical section */;
   exitcritical (Ra);
   /* following code */;
  }
}
```

Mutual Exclusion Tools

互斥实现工具

西南交通大学 Southwest Jiaotong University

- **Software Solution with busy waiting technique:**
  - Dekker algorithm
  - Peterson algorithm

基于忙等待的软件

- **Hardware support**

硬件支持

  - Interrupt (uniprocesor)
  - Machine instruction (MPS)
- **Supported by OS or enforced by Language compilers without busy waiting**
  - Semaphore
  - Monitor
  - message

信号量 操作系统/语言编译器支持

5.2 Software Solution with busy waiting technique

- e.g. Only one access to a memory location can be made at a time

```
   /* PROCESS 0 /*                  /* PROCESS 1 *
.                                  .
.          Busy waiting            .
while (turn != 0)                  while (turn != 1)
   /* do nothing */ ;                 /* do nothing */;
/* critical section*/;              /* critical section*/;
turn = 1;                          turn = 0;
.                                  .
```

(a) First attempt

```
      /* PROCESS 0 *                  /* PROCESS 1 *
   .                                  .
   .                                  .
   while (flag[1])                    while (flag[0])
      /* do nothing */;                  /* do nothing */;
   flag[0] = true;                    flag[1] = true;
   /*critical section*/;              /* critical section*/;
   flag[0] = false;                   flag[1] = false;
   .                                  .
```

(b) Second attempt

```
int   turn = 0;
```

- The pace of execution is dictated by the slower of the two processes
- If one process fails, the other process is permanently blocked.

```
enum      boolean (false = 0; true = 1);
boolean   flag[2] = 0, 0
```

- if a process fails inside its critical section or after setting its flag to true just, other process is permanently blocked
- it does not even guarantee mutual exclusion

Software Solution with busy waiting technique

西南交通大学 Southwest Jiaotong University

```
/* PROCESS 0 *                    /* PROCESS 1 *
.                                 .
.                                 .
flag[0] = true;                   flag[1] = true;
while (flag[1])                   while (flag[0])
   /* do nothing */;                  /* do nothing */;
/* critical section*/;             /* critical section*/;
flag[0] = false;                  flag[1] = false;
.
```

(c) Third attempt

```
/* PROCESS 0 *                    /* PROCESS 1 *
.                                 .
.                                 .
flag[0] = true;                   flag[1] = true;
while (flag[1]) {                 while (flag[0]) {
   flag[0] = false;                  flag[1] = false;
   /*delay */;                      /*delay */;
   flag[0] = true;                   flag[1] = true;
}                                 }
/*critical section*/;             /* critical section*/;
flag[0] = false;                  flag[1] = false;
.
```

(d) Fourth attempt

引入"谦让机制"

```
enum     boolean (false = 0; true = 1);
boolean  flag[2] = 0, 0
```

- If both processes set their flags to true before either has executed the **while** statement, then each will think that the other has entered its critical section, causing deadlock.

死锁

P0 sets `flag[0]` to true.
Pl sets `flag[1]` to true.
P0 checks `flag[1]`.
Pl checks `flag[0]`.
P0 sets `flag[0]` to false.
Pl sets `flag[1]` to false.
P0 sets `flag[0]` to true.
Pl sets `flag[1]` to true.

extended indefinitely

Livelock（活锁）

- possible sequences of executions that succeed,
- one or more execution sequences in which no process ever enters its critical section.

永无进程进入临界区

Peterson's Algorithm 假特森算法

西南交通大学 Southwest Jiaotong University

boolean flag [2];
int turn;

```
void main()
{
        flag [0] = false;
        flag [1] = false;
        parbegin  (P0, P1);
}
```

并发执行!

```
void P0()
{
      while (true) {
            flag [0] = true; // 声明"我要进!"
            turn = 1; // 谦让
            while (flag [1] && turn == 1) /* do nothing */;
            /* critical section */;
            flag [0] = false;
            /* remainder */;
      }
}
```

等

```
void P1()
{
      while (true) {
            flag [1] = true;
            turn = 0;
            while (flag [0] && turn == 0) /* do nothing */;
            /* critical section */;
            flag [1] = false;
            /* remainder */
      }
}
```

Mutual Exclusion: Hardware Support

西南交通大学 Southwest Jiaotong University

Interrupt Disabling

- disabling interrupts guarantees mutual exclusion in uniprocessor system
- Not work in multiprocessor

Disadvantages:

- the efficiency of execution could be noticeably degraded

```
while (true) {
    /* disable interrupts */;
    /* critical section */;
    /* enable interrupts */;
    /* remainder */;
}
```

5.2 Mutual Exclusion: Hardware Support

西南交通大学 Southwest Jiaotong University

Special Machine Instructions 专用机器指令

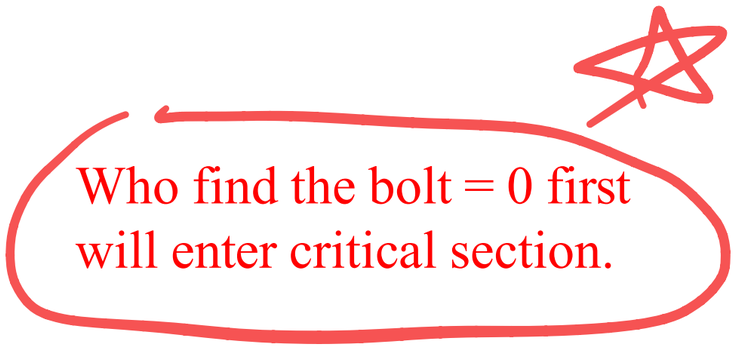
- Compare&Swap Instruction 比较交换指令
  - also called a “compare and exchange instruction”
  - a compare is made between a memory value and a test value 将内存值与测试值比较
  - if the values are the same a swap occurs 若值相等，则交换
  - carried out atomically 原子性执行
- exchanges instruction
  - Exchange the contents of a register with that of a memory location.

寄存器 内存单元

Compare and Swap Instruction

西南交通大学 Southwest Jiaotong University

```
int compare_and_swap (int *word, int
testval, int newval)
{
int oldval;
oldval = *word
if (oldval == testval) *word = newval;
return oldval;
}
```




```
/* program  mutualexclusion */
const int n = /* number of processes */;
int bolt;
void P(int  i)
{
    while (true)  {
     while (compare_and_swap(bolt, 0, 1) == 1)
          /* do nothing */;
      /* critical section */;
      bolt = 0;
      /* remainder */;
    }
}
void main() {
    bolt = 0;
    parbegin (P(1), P(2), ... ,P(n));
}
```

Exchange Instruction


```
void exchange (int *register,  int *memory)
{
    int temp;
    temp = *memory;
    *memory = *register;
    *register = temp;
}
```

Who find the bolt = 0 first will enter critical section.

```
/* program mutualexclusion */
int const n = /* number of processes */;
int bolt;
void P(int  i)
{
    while (true)
        int  keyi = 1;
        do  exchange (&keyi, &bolt)
        while  (keyi != 0);
        /* critical section */;
        bolt = 0;
        /* remainder */;
    }
}
void main() {
    bolt = 0;
    parbegin  (P(1), P(2), ..., P(n));
}
```

Special Machine Instruction

西南交通大学 Southwest Jiaotong University

Advantages

- Applicable to any number of processes on either a single processor or multiple processors sharing main memory
- Simple and easy to verify
- It can be used to support multiple critical sections; each critical section can be defined by its own variable

Disadvantages

- Busy-waiting is employed, thus while a process is waiting for access to a critical section it continues to consume processor time
- Starvation is possible when a process leaves a critical section and more than one process is waiting to access memory
- Deadlock is possible: P1 is then interrupted to give the processor to P2, which has higher priority. If P2 now attempts to use the same resource as P1, it will be denied access because of the mutual exclusion mechanism.

Common Concurrency Mechanisms by OS/Programming Language

西南交通大学 Southwest Jiaotong University

Semaphore	An integer value used for signaling among processes. Only three operations may be performed on a semaphore, all of which are atomic: initialize, decrement, and increment. The decrement operation may result in the blocking of a process, and the increment operation may result in the unblocking of a process. Also known as a **counting semaphore** or a **general semaphore**.
Binary semaphore	A semaphore that takes on only the values 0 and 1.
Mutex	Similar to a binary semaphore. A key difference between the two is that the process that locks the mutex (sets the value to 0) must be the one to unlock it (sets the value to 1).
Condition variable	A data type that is used to block a process or thread until a particular condition is true.
Monitor	A programming language construct that encapsulates variables, access procedures, and initialization code within an abstract data type. The monitor's variable may only be accessed via its access procedures and only one process may be actively accessing the monitor at any one time. The access procedures are *critical sections*. A monitor may have a queue of processes that are waiting to access it.
Event flags	A memory word used as a synchronization mechanism. Application code may associate a different event with each bit in a flag. A thread can wait for either a single event or a combination of events by checking one or multiple bits in the corresponding flag. The thread is blocked until all of the required bits are set (AND) or until at least one of the bits is set (OR).
Mailboxes/messages	A means for two processes to exchange information and that may be used for synchronization.
Spinlocks	Mutual exclusion mechanism in which a process executes in an infinite loop waiting for the value of a lock variable to indicate availability.

5.3 Semaphore

西南交通大学 Southwest Jiaotong University

- **A variable that has an integer value upon which only three operations are defined:**
  - May be initialized to a nonnegative integer value
  - The semWait operation decrements the value
  - The semSignal operation increments the value

- **There is no way to inspect or manipulate semaphores other than these three operations**

- **Consequences**
  - There is no way to know before a process decrements a semaphore **whether it will block or not**
  - There is no way to know **which process will continue** immediately on a uniprocessor system when two processes are running concurrently
  - You don’t know **whether another process is waiting** so the number of unblocked processes may be zero or one

Semaphore Primitives

西南交通大学
Southwest Jiaotong University

```
struct semaphore {
        int count;
        queueType queue;
};
void semWait(semaphore s)
{
        s.count--;
        if (s.count < 0) {
          /* place this process in s.queue */;
          /* block this process */;
        }
}
void semSignal(semaphore s)
{
        s.count++;
        if (s.count<= 0) {
          /* remove a process P from s.queue */;
          /* place process P on ready list */;
        }
}
```

Binary Semaphore Primitives


```
struct binary_semaphore {
        enum {zero, one} value;
        queueType queue;
};
 void semWaitB(binary_semaphore s)
{
        if (s.value == one)
            s.value = zero;
        else {
                    /* place this process in s.queue */;
                    /* block this process */;
        }
}
 void semSignalB(semaphore s)
{
        if (s.queue is empty())
            s.value = one;
        else {
                    /* remove a process P from s.queue */;
                    /* place process P on ready list */;
        }
}
```

申请信号量 → 等待操作=P操作

释放信号量 → 唤醒操作=V操作

Strong/Weak Semaphores

- A queue is used to hold processes waiting on the semaphore

采用队列

Strong Semaphores 强 Semaphores

- the process that has been blocked the longest is released from the queue first (FIFO)

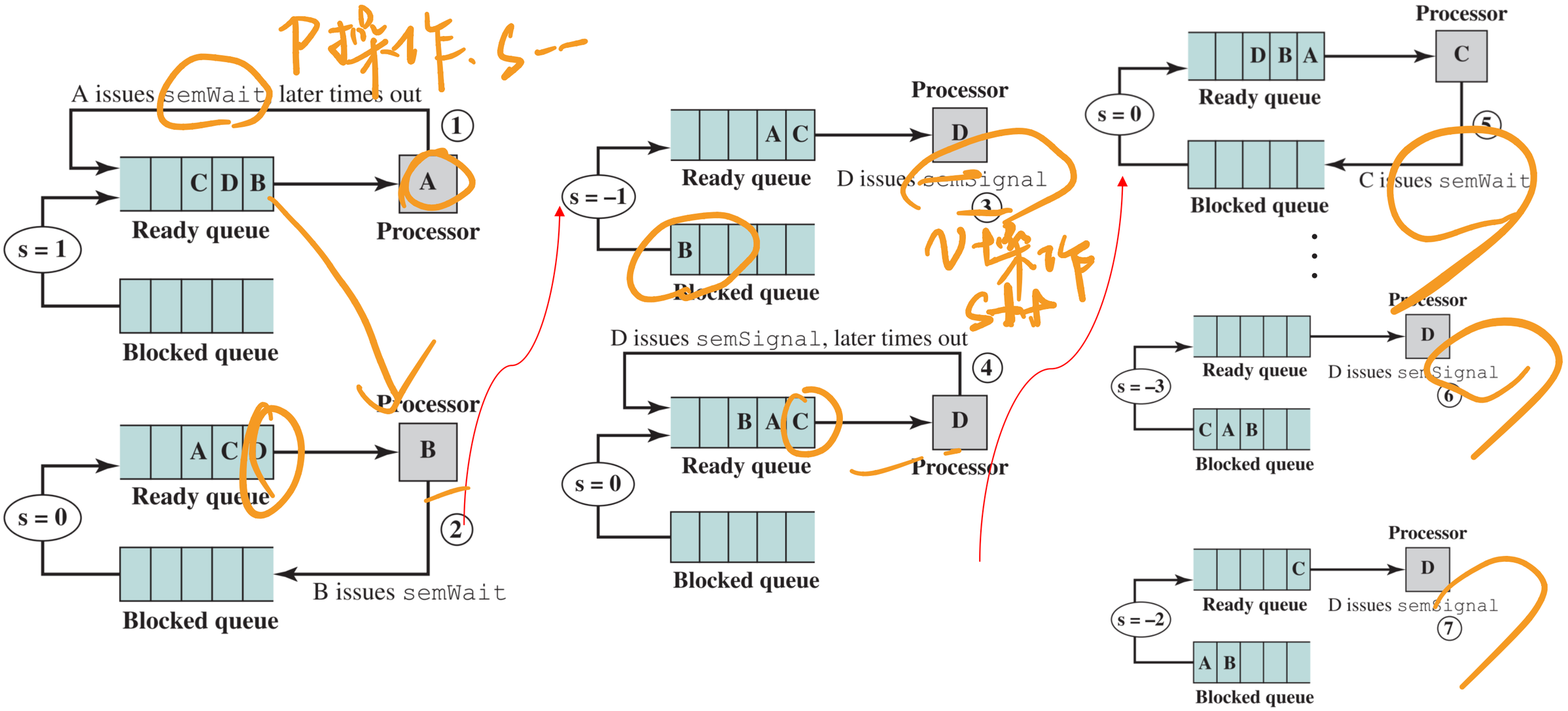
先来先服务

Weak Semaphores 弱

- the order in which processes are removed from the queue is not specified

无特定顺序

Example of Semaphore Mechanism



Mutual Exclusion Using Semaphores

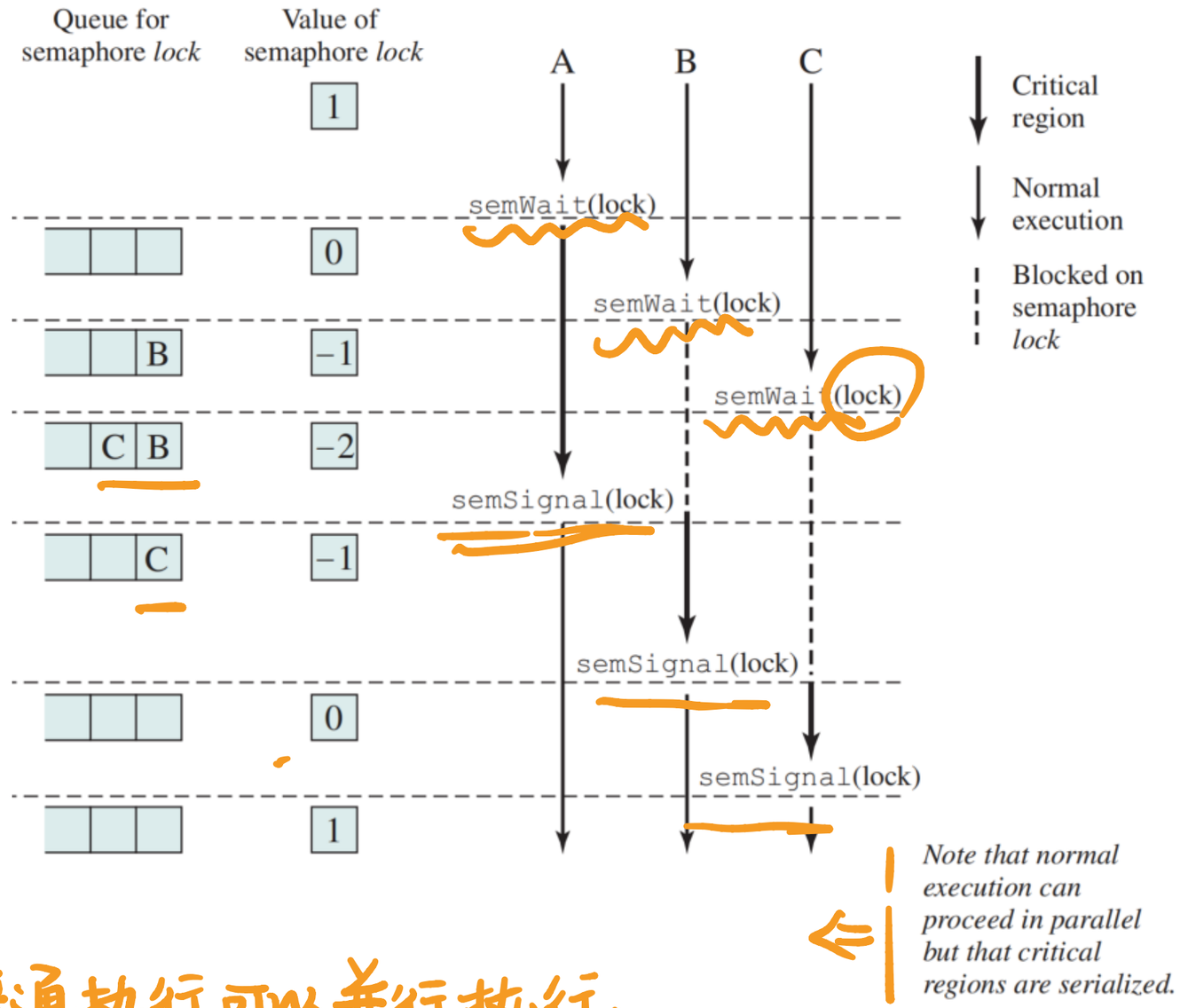
西南交通大学 Southwest Jiaotong University

```
/*  program mutualexclusion */
const int n = /* number of processes */;
semaphore s = 1;
void P(int  i)
{
     while (true) {
          semWait(s);
          /* critical section */;
          semSignal(s);
          /* remainder */;
     }
}
void main()
{
     parbegin (P(1), P(2), . . . , P(n));
}
```

初始化信号量

Shared Data Protected by a Semaphore

- *s.count>=0,* is the number of processes that can execute semWait (*s*)
- The magnitude of *s.count<0* is the number of processes suspended in *s.queue*.



进程的普通执行可以并行执行.
但在临界区是串行(互斥)访问的

Producer/Consumer Problem

生产者/消费者问题

Situation:

- one or more producers are generating data and placing these in a buffer
- a single consumer is taking items out of the buffer one at time
- only one producer or consumer may access the buffer at any one time

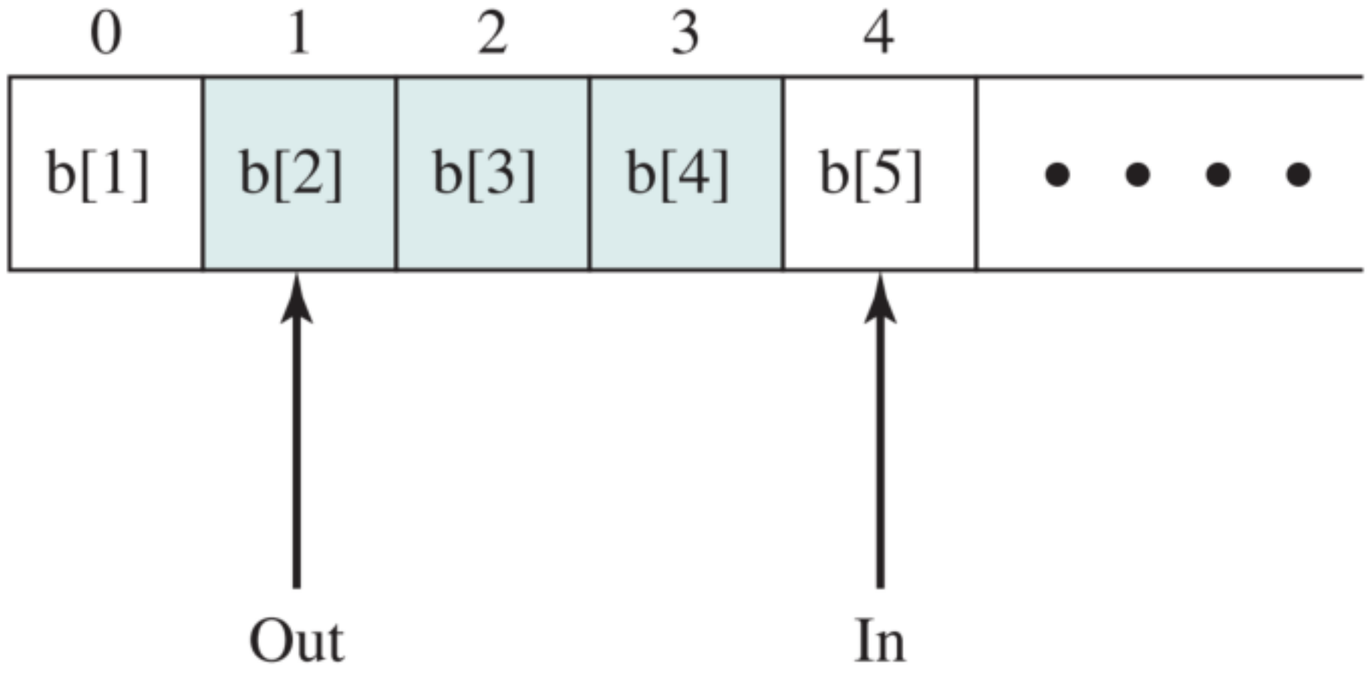
缓冲区

Problem:

- Mutual exclusion-one process visit the buffer
- Synchronization- producer can't add data into full buffer
- Synchronization- consumer can't remove data from an empty buffer

互斥 同步 同步

Buffer Structure



Note: Shaded area indicates portion of buffer that is occupied.

Infinite Buffer for the Producer/Consumer Problem

An Incorrect Solution

西南交通大学 Southwest Jiaotong University

```
/*  program producerconsumer */
      int n;
      binary_semaphore s = 1, delay = 0;
      void producer()
      {
            while (true) {
                  produce();
                  semWaitB(s);
                  append();
                  n++;
                  if (n==1) semSignalB(delay);
                  semSignalB(s);
            }
      }
      void consumer()
      {
            semWaitB(delay);
            while (true) {
                  semWaitB(s);
                  take();
                  n--;
                  semSignalB(s);
                  consume();
                  if (n==0) semWaitB(delay);
            }
      }
      void main()
      {
            n = 0;
            parbegin (producer, consumer);
      }
```

Possible Scenario

	Producer	Consumer	s	*n*	Delay
1			1	0	0
2	`semWaitB(s)`		0	0	0
3	`n++`		0	1	0
4	**`if`** `(n==1)` `(semSignalB(delay))`		0	1	1
5	`semSignalB(s)`		1	1	1
6		`semWaitB(delay)`	1	1	0
7		`semWaitB(s)`	0	1	0
8		`n--`	0	0	0
9		`semSignalB(s)`	1	0	0
10	`semWaitB(s)`		0	0	0
11	`n++`		0	1	0
12	**`if`** `(n==1)` `(semSignalB(delay))`		0	1	1
13	`semSignalB(s)`		1	1	1
14		**`if`** `(n==0)` `(semWaitB(delay))`	1	1	1
15		`semWaitB(s)`	0	1	1
16		`n--`	0	0	1
17		`semSignalB(s)`	1	0	1
18		**`if`** `(n==0)` `(semWaitB(delay))`	1	0	0
19		`semWaitB(s)`	0	0	0
20		`n--`	0	−1	0
21		`semSignalB(s)`	1	−1	0

Remove empty produce

Correct Solution

西南交通大学
Southwest Jiaotong University

```
/* program producerconsumer */
      int n;
      binary_semaphore s = 1, delay = 0;
      void producer()
      {
            while (true) {
                  produce();
                  semWaitB(s);
                  append();
                  n++;
                  if (n==1) semSignalB(delay);
                  semSignalB(s);
            }
      }
      void  consumer()
      {
            int  m; /* a local variable */
            semWaitB(delay);
            while  (true) {
                  semWaitB(s);
                  take();
                  n--;
                  m = n;
                  semSignalB(s);
                  consume();
                  if  (m==0) semWaitB(delay);
            }
      }
      void  main()
      {
            n = 0;
            parbegin  (producer, consumer);
      }
```

A Solution by Using Semaphores

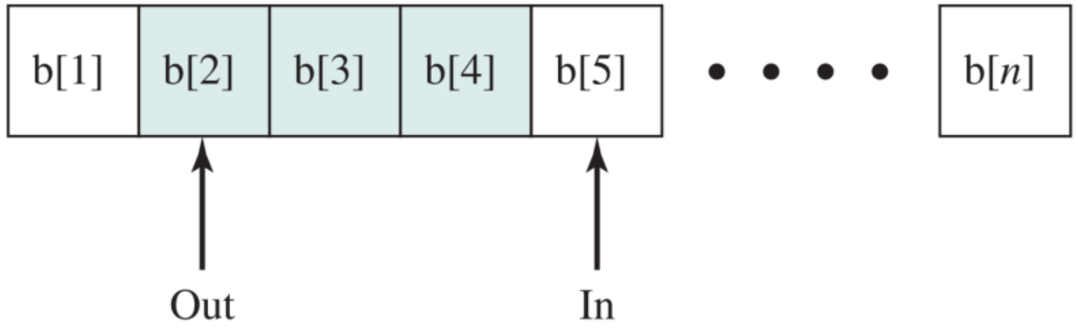
西南交通大学 Southwest Jiaotong University

```
/* program producerconsumer */
     semaphore n = 0, s = 1;
     void producer()
     {
          while (true) {
                produce();
                semWait(s);
                append();
                semSignal(s);
                semSignal(n);
          }
     }
     void consumer()
     {
          while (true) {
                semWait(n);
                semWait(s);
                take();
                semSignal(s);
                consume();
          }
     }
     void main()
     {
               parbegin  (producer, consumer);
     }
```

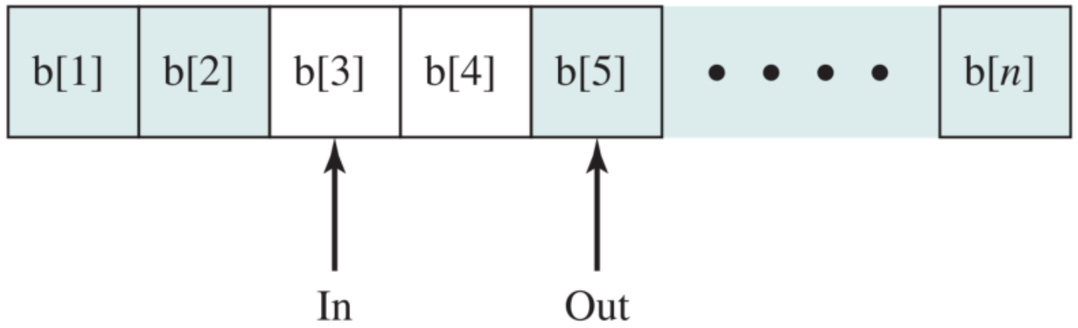
A mistake is made and the operations semSignal(s) and semSignal(n) are interchanged, Would this affect the program?

Finite Circular Buffer

Block on:	Unblock on:
Producer: insert in full buffer	Consumer: item inserted
Consumer: remove from empty buffer	Producer: item removed



(a)



(b)

A Solution by Using Semaphores

```
/* program boundedbuffer */
      const int sizeofbuffer = /* buffer size */;
      semaphore s = 1, n = 0, e = sizeofbuffer;
      void producer()
      {
            while (true) {
                  produce();
                  semWait(e); 申一个空位(P(e))
                  semWait(s); 申一个锁(P(s))
                  append();
                  semSignal(s); 开一个锁(V(s))
                  semSignal(n); 放一个空位(V(n))
            }
      }
      void consumer()
      {
            while  (true) {
                  semWait(n);
                  semWait(s);
                  take();
                  semSignal(s);
                  semSignal(e);
                  consume();
            }
      }
      void main()
      {
            parbegin  (producer, consumer);  并行
      }
```

空区计数

互斥锁

满缓冲区计数

Readers/Writers Problem

西南交通大学 Southwest Jiaotong University

- **A data area is shared among many processes**
  - some processes only read the data area (readers) 只读
  - some only write to the data area (writers) 可写(可编辑)

- **Conditions that must be satisfied:**
  - any number of readers may simultaneously read the file 可多人同时读
  - only one writer at a time may write to the file 仅允许一人写
  - if a writer is writing to the file, no reader may read it

若有人写、则不可有人读

Readers Have Priority

读者优先

```
/* program readersandwriters */
int readcount;
semaphore x = 1,wsem = 1;
void reader()
{
    while (true){
     semWait (x);
     readcount++;
     if(readcount == 1)
         semWait (wsem);
     semSignal (x);
     READUNIT();
     semWait (x);
     readcount--;
     if(readcount == 0)
         semSignal (wsem);
     semSignal (x);
     }
}
void writer()
{
      while (true){
       semWait (wsem);
       WRITEUNIT();
       semSignal (wsem);
      }
}
void  main()
{
     readcount = 0;
     parbegin (reader,writer);
}
```

x: synchronization (S) semaphore

wsem： mutual exclusion (ME) semaphore

申请信号量 互斥信号量

S-x

mutual exclusion with writers

The writer must wait for all the reader completing the reading, which may lead to starvation

mutual exclusion with writers

释放信号量

导致写者饥饿

读的人太多

Writers Have Priority

int readcount,writecount; semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;
void main()
{readcount = writecount = 0;
parbegin (reader, writer);}

x，y: synchronization(S) semaphores
wsem, rsem: mutual exclusion (ME) semaphores

并行起动写者和读者

```
void reader()
{
    while (true){
        semWait (z);
            semWait (rsem);
                semWait (x);
                    readcount++;
                    if (readcount == 1)
                        semWait (wsem);
                    semSignal (x);
                semSignal (rsem);
            semSignal (z);
            READUNIT();
            semWait (x);
                readcount--;
                if (readcount == 0) semSignal (wsem);
            semSignal (x);
    }
}
```

S-x

ME-rsem

ME-wsem

Readers are stopped at Z queue, rsem={1, 0}, z={1, 0, -1,…}. the writer only waits one reader's reading job

```
void writer ()
{
    while (true){
        semWait (y);
            writecount++;
            if (writecount == 1)
                semWait (rsem);
        semSignal (y);
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
            writecount--;
            if (writecount == 0) semSignal (rsem);
        semSignal (y);
    }
}
```

S-y

ME-wsem

ME-rsem

Only the first writer can be stopped by reader. Others are stopped by writers.

Readers wait for all the writers completing

读者必须等全部写者写完

Problem

- (Readers/Writers problem) At 1:00pm, a writer W1 begins writing. At 1:30pm, the reader R1 arrives and tries to read, but fails because W1 is writing. At 2:00pm, readers R2, R3 and R4 arrive. The writer W1 finishes writing and leaves at 3:00pm. Each reading / writing operation takes 2 hours.
- Trace what happens to the readers when they arrive (e.g. where they are blocked), and how all the readers are unblocked at 3:00pm when the writer finishes writing. Trace the readers until all of them are reading.

下午1:00，写者W1开始写入
下午1:30，读者R1到达并尝读取
2:00 读者R2、R3、R4到达
3:00 写者W1完成写入
每次读/写均耗时2个小时

```
    /* reader thread */                          /* writer thread */
    repeat                                       repeat
a1    wait(x);                                   b1   wait(wsem);
a2    readcount = readcount+1;                   b2     WRITE;
a3    if (readcount==1) wait(wsem);              b3   signal(wsem);
a4  signal(x);                                   forever
a5  READ;
a6  wait(x);
      readcount = readcount-1;
      if (readcount==0) signal(wsem);
    signal(x);
forever;
```

// 若是写者优先，则在此加P(w)
V(w)
// 最后一个读者释放写锁

Solution

下午1:00，写者W1开始写入
下午1:30，读者R1到达并尝读败
2:00 读者R2、R3、R4到达
3:00 写者W1完成写入
每次读/写均耗时2个小时

			Readcount	X (信号量)	wsem	
			0	1	1	
1:00	W1	b1	0	1	0	
		b2				WRITE file
1:30	R1	a1	0	0	0	
		a2	1	0	0	
		a3	1	0	-1	Blocked
2:00	R2					
	R3					
	R4					
3:00	W1					
	R1					
	R1					
	R2					
	R2					
	R2					
	R2					
	R2					
	R3					
	R3					
	R3					
	R3					
	R4					
	R4					
	R4					
	R4					

申请读计数的锁

wait(x).
读者数变为1. readcount+1

第1个读者独拍占wsem

Solution

			Readcount	X (信号量)	wsem	
			0	1	1	
1:00	W1	b1	0	1	0	
		b2				WRITE file
1:30	R1	a1	0	0	0	
		a2	1			
		a3			-1	Blocked
2:00	R2			-1		
	R3			-2		
	R4			-3		
3:00	W1					
	R1			-2		
	R1					read
	R2		2			
	R2					
	R2			-1		
	R2					read
	R3		3			
	R3					
	R3			0		
	R3					read
	R4		4			
	R4					
	R4			1		
	R4					read

```
   /* reader thread */                    /* writer thread */
   repeat                                 repeat
a1   wait(x);                         b1   wait(wsem);
a2   readcount = readcount+1;         b2     WRITE;
a3   if (readcount==1) wait(wsem);    b3   signal(wsem);
a4   signal(x);                        forever
a5   READ;
a6   wait(x);
     readcount = readcount-1;
     if (readcount==0) signal(wsem);
   signal(x);
forever;
```

			Readcount	X (信号量)	wsem	
	R2					
	R3					
	R4					
5:00	**R1**			-3		
	R1		3			
	R1					
	R1			-2		
	R2		2			
	R2					
	R2			-1		
	R3		1			
	R3					
	R3			0		
	R4		0			
	R4				1	
	R4			1		

Monitors (管程)

西南交通大学
Southwest Jiaotong University

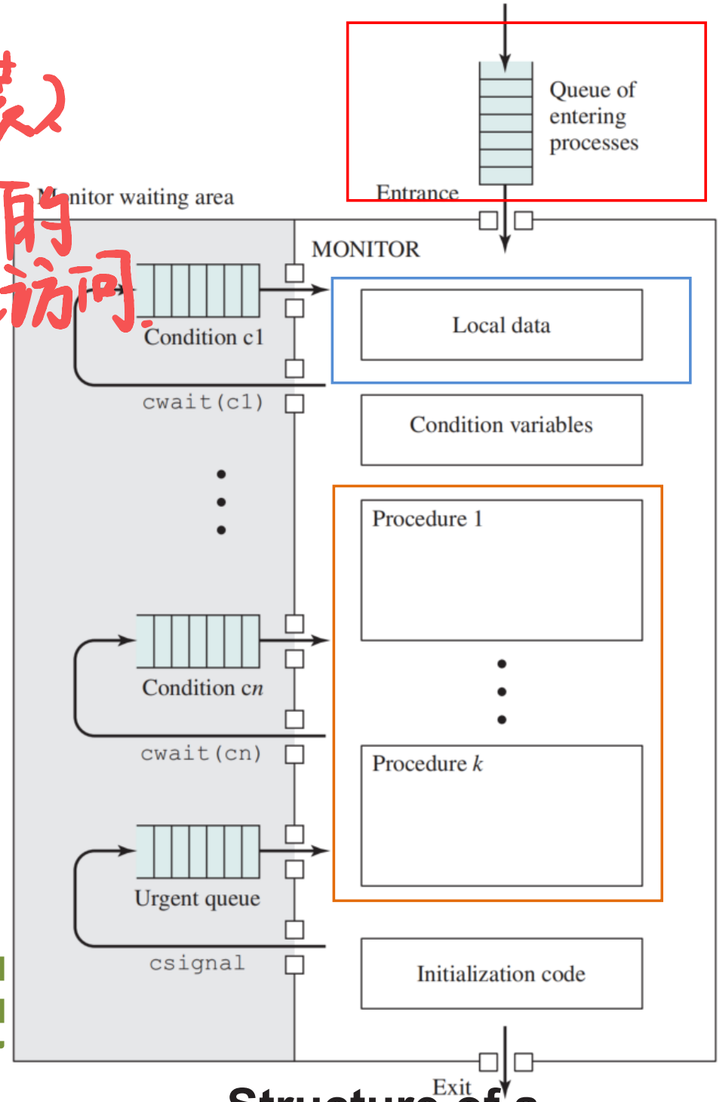
- A Programming language construct that provides equivalent functionality to that of semaphores and is easier to control
- Implemented in many programming languages
  - including Concurrent Pascal, Pascal-Plus, Modula-2, Modula-3, and Java
- （Hoare'version）Software module consisting of
  - one or more procedures,
  - an initialization sequence,
  - and local data

Monitor Characteristics

- **Local data variables**: are accessible only by the **monitor's procedures** and not by any external procedure.
- **Procedures**: Process **enters** monitor by **calling** one of monitor's procedures

Mutual exclusion : **Only one process may be executing** in the monitor and **have access to** the local data variables at a time. Others are **blocked**.

What if a process is blocked in the monitor?



Structure of a Monitor

Synchronization 同步

- **Condition variables**: **contained within the monitor and accessible only within the monitor**

条件变量

- **a special data type in monitor, are operated on by two functions:**
  - cwait(c): block the executing process on condition c; the monitor can be used be the other processes.
  - csignal(c): resume one of process blocked by condition cwait(c); **if there is no such process, do nothing and signal is lost.**

唤醒量

Note that monitor wait and signal operations are different from those for the semaphore.

管程的等待与唤醒操作与信号量不同

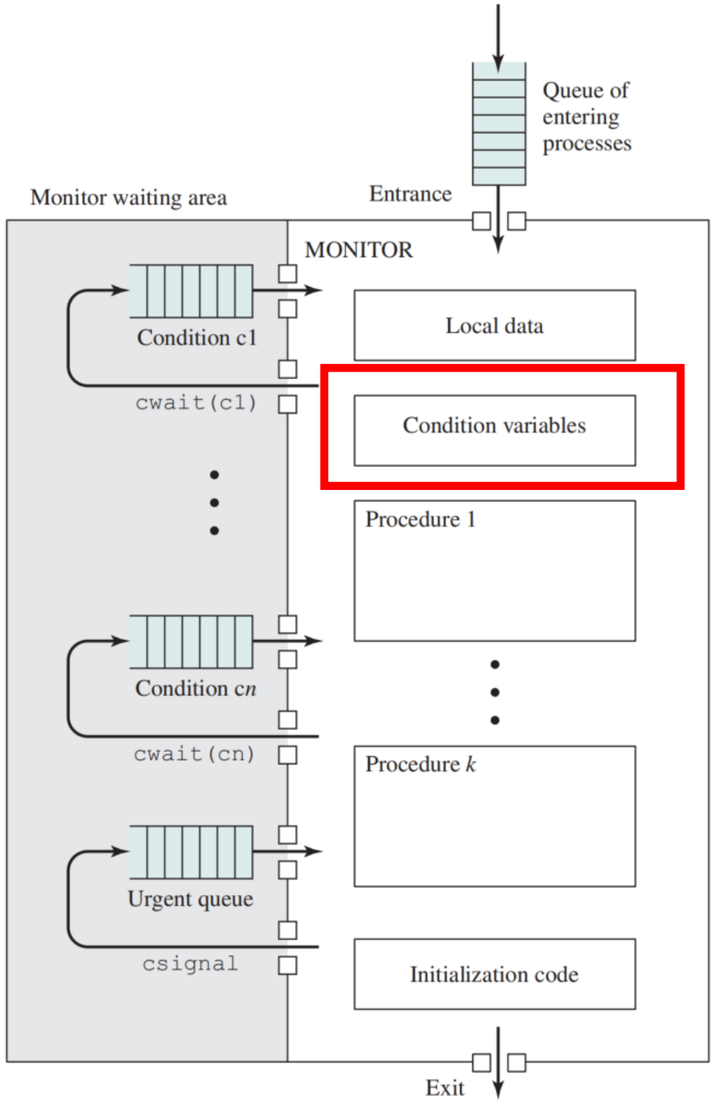


Figure 5.15 Structure of a Monitor

Producer and Consumer Solution via Monitor

西南交通大学 Southwest Jiaotong University

```
void producer()
{
      char x;
      while (true) {
      produce(x);
      append(x);
      }
}
void consumer()
{
      char x;
      while (true) {
      take(x);
      consume(x);
      }
}
void main()
{
      parbegin (producer, consumer);
}
```

Request to enter monitor via monitor's procedures

```
/* program producerconsumer */
monitor boundedbuffer;
char buffer [N];                                    /* space for N items */
int nextin, nextout;                                /* buffer pointers */
int count;                                          /* number of items in buffer */
cond notfull, notempty;                 /* condition variables for synchronization */
void append (char x)

{
      if (count == N) cwait(notfull);           /* buffer is full; avoid overflow */
      buffer[nextin] = x;
      nextin = (nextin + 1) % N;
      count++;
      /* one more item in buffer */
      csignal (notempty);                       /*resume any waiting consumer */
}
void take (char x)
{
      if (count == 0) cwait(notempty);        /* buffer is empty; avoid underflow */
      x = buffer[nextout];
      nextout = (nextout + 1) % N;
      count--;                                  /* one fewer item in buffer */
      csignal (notfull);                        /* resume any waiting producer */
}
{                                                           /* monitor body */
      nextin = 0; nextout = 0; count = 0;           /* buffer initially empty */
}
```

Can the Producer/consumer visit buffer directly?

不能，必须通过调用管程的过程间接访问

Comparison with Semaphores

西南交通大学 Southwest Jiaotong University

- **Disadvantages：it is possible to make mistakes in the synchronization function of monitors.**
  - For example, if either of the **csignal functions** in the boundedbuffer monitor are **omitted**, then processes entering the corresponding **condition queue** are **permanently hung up**.
- **Advantages:**
  - all of the synchronization functions are confined to the monitor. Therefore, it is **easier to verify that the synchronization** has been done correctly and to **detect bugs**.
  - once a monitor is correctly programmed, access to the protected resource is correct for access from all processes.
- **In contrast, with semaphores, resource access is correct only if all of the processes that access the resource are programmed correctly**

Message Passing 消息传递

西南交通大学 Southwest Jiaotong University

- **Motivation——when processes interact with one another, two fundamental requirements must be satisfied:**
  - Synchronization: Processes need to be synchronized to enforce mutual exclusion; 同步
  - Communication: cooperating processes may need to exchange information; 通信

- **Message Passing is one approach to provide both of these functions**
  - works in **distributed systems** and **shared memory multiprocessor** and **uniprocessor systems**

Message Passing

- Typically Features：
- The actual function is normally provided in the form of a pair of primitives (原语):
  - **send** (destination, message)
  - **receive** (source, message)
- A process sends information in the form of a message to another process designated by a destination
- A process receives information by executing the receive primitive, indicating the source and the message

- This is the minimum set of operations needed for processes to engage in message passing.

Design Issues

西南交通大学
Southwest Jiaotong University

Synchronization
- Send
  - blocking
  - nonblocking
- Receive
  - blocking
  - nonblocking
  - test for arrival

Addressing
- Direct
  - send
  - receive
    - explicit
    - implicit
- Indirect
  - static
  - dynamic
  - ownership

Format
- Content
- Length
  - fixed
  - variable

Queueing Discipline
- FIFO
- Priority

Design Characteristics of Message Systems for Interprocess Communication and Synchronization

Synchronization 同步性

西南交通大学 Southwest Jiaotong University

- **The receiver cannot receive a message until it has been sent by another process (synchronization)**
- **When a send primitive is executed in a process there are two possibilities:** 原语
  - The sending process is **blocked** until the message is received
  - The sending process is **not blocked**
- **When a receive primitive is executed in a process there are two possibilities:**
  - if a message has previously been sent the message is received and execution continues (**unblocked**)
  - if there is no waiting message, (a) the process is **blocked** until a message arrives or (b) the process continues to execute, abandoning the attempt to receive (**unblocked**)

Three Common Combinations

西南交通大学 Southwest Jiaotong University

- **Blocking send, blocking receive (e.g., Printer):** 打印机
  - **Both the sender and receiver are blocked** until the message is delivered;
  - this is sometimes referred to as a **rendezvous (会合)**.
  - This combination allows for **tight synchronization** between processes.

紧密同步

- **Nonblocking send, blocking receive (e.g., service request)——probably the most useful combination.**
  - The **receiver is blocked** until the requested message arrives
  - It allows a **process to send one or more messages** to a **variety of destinations** as quickly as possible.

服务请求

多个目标

- **Nonblocking send, nonblocking receive (msg loss):**
  - Neither party is required to wait.

消息丢失

Addressing 寻址

- **Addressing: which process is to receive the message (send)/ indicate the source of a message (receive)**

发送时指定接收消息的进程
接收时指明消息的源进程

- **Direct addressing**
  - Send primitive includes a specific identifier (PID) of the destination process
  - Receive primitive can be handled : 显式寻址
    - require that the process explicitly designate a sending process (PID), effective for cooperating concurrent processes 源参数
    - implicit addressing, source parameter of the receive primitive possesses a value returned when the receive operation has been performed (e.g., Printer)

隐式寻址 返回值

Indirect Process

西南交通大学 Southwest Jiaotong University

Indirect addressing

- Messages are not sent directly from sender to receiver
- Messages are sent to a **shared data structure** consisting of **queues** that can temporarily hold messages

信箱 共享数据结构的队列

Queues are referred to as mailboxes. For two processes to communicate:

- **one process** sends a message to the appropriate mailbox;
- **the other process** picks up the message from the mailbox.
- Allows for **greater flexibility** in the use of messages, The relationship between senders and receivers can be:

灵活性

  - **one-to-one**:
  - **many-to-one**:
  - **one-to-many**:
  - **Many-to-many**:

Indirect Process Communication

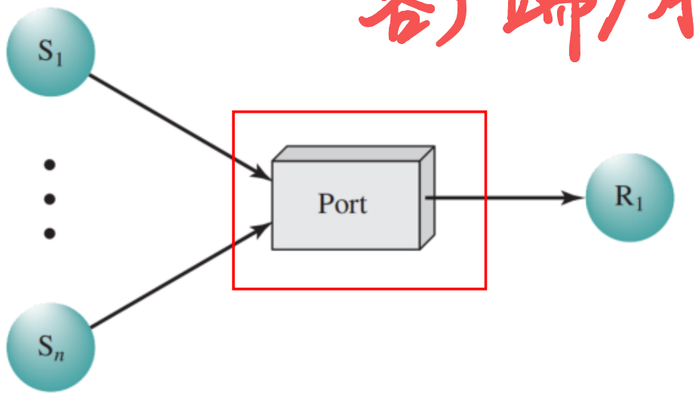
西南交通大学 Southwest Jiaotong University

private communications

(a) One-to-one

client/server interaction

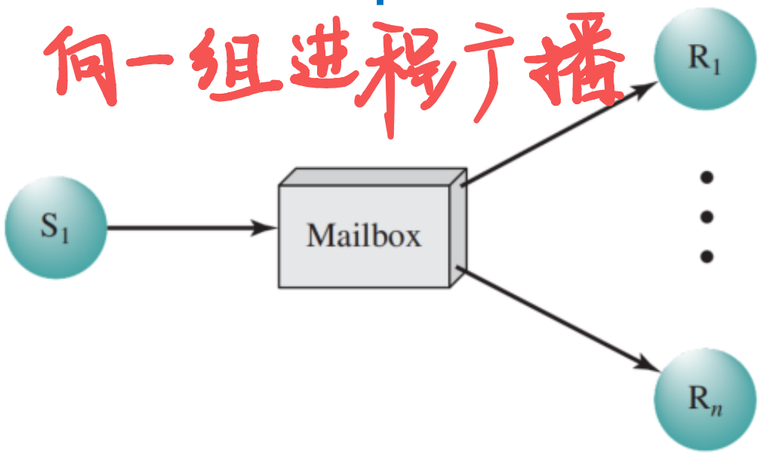
客户端/服务器交互



(b) Many-to-one

broadcast to a set of processes

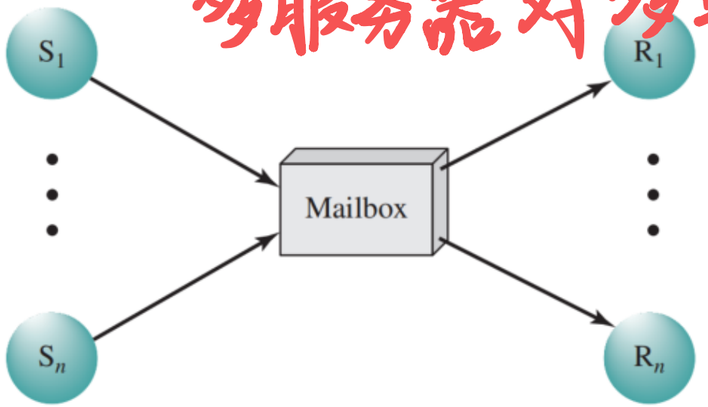
向一组进程广播



(c) One-to-many

multi-server provides services to multi-clients

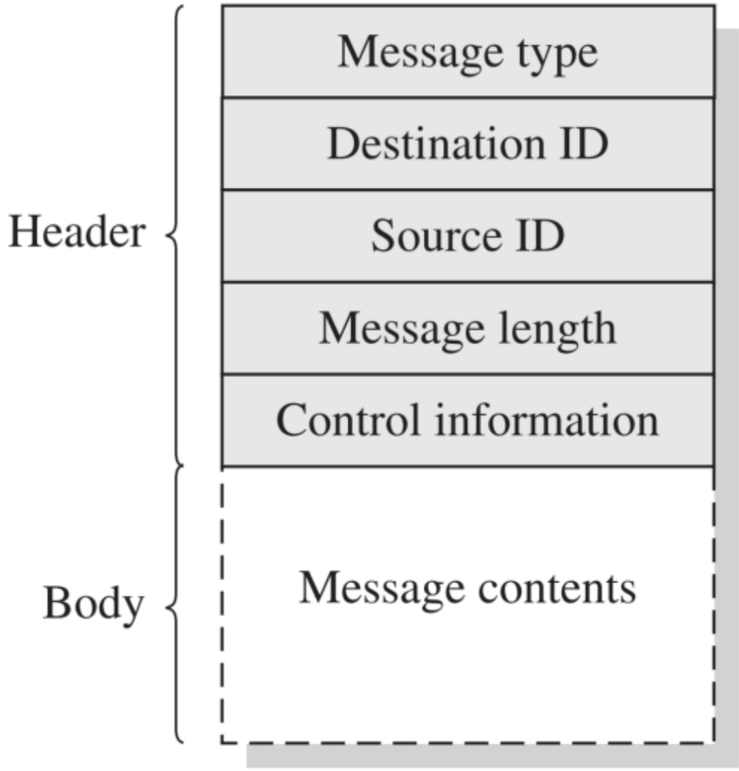
多服务器对多客户端



(d) Many-to-many

General Message Format

- **Short, fixed-length messages to save the processing and storage overhead;**
- **A file for a large amount of data exchange;**
- **Variable-length messages:**
  - **Header**
    - **control information**: a pointer field so that a linked list of messages can be created; a sequence number, to keep track of the number and order of messages passed between source and destination;
    - a priority field.
  - **body**



General Message Format for Variable-length messages

Mutual Exclusion Using Messages

西南交通大学 Southwest Jiaotong University

- **blocking receive primitive:**
  - If the mailbox is **empty**, then the process is **blocked**.
  - Once a process has acquired the message, it performs its critical section then **places the message back** into the mailbox.
- **the nonblocking send primitive.**

非阻塞发送原语

接收到消息后，先执行临界区代码，再放回信箱

- the message is as a **token** passed from process to process.
- **Only one process** can access the critical section via receiving the msg

令元

```
/* program mutualexclusion */
const int n = /* number of process */
void P(int i)
{
        message msg;
        while (true) {
            receive (box, msg);
            /* critical section */;
            send (box, msg);
            /* remainder */;
        }
}
void main()
{
        create mailbox (box);
        send (box, null);
        parbegin (P(1), P(2), . . . , P(n));
}
```

A set of concurrent processes share a mailbox.

A Solution to the Bounded-Buffer Producer/Consumer Problem Using Messages

西南交通大学 Southwest Jiaotong University

```
const int
    capacity = /* buffering capacity */ ;
    null = /* empty message */ ;
int i;
void producer()
{   message pmsg;
   while (true) {
     receive (mayproduce,pmsg);
     pmsg = produce();
     send (mayconsume,pmsg);
    }
}
void consumer()
{   message cmsg;
   while (true) {
     receive (mayconsume,cmsg);
     consume (cmsg);
     send (mayproduce,null);
    }
}
void main()
{
    create_mailbox (mayproduce);
    create_mailbox (mayconsume);
   for (int i = 1;i<= capacity;i++) send (mayproduce,null);
    parbegin (producer,consumer);
}
```

mayconsume mailbox is buffer section

The number of message in mayproduce decreases with each production

The number of message in mayproduce grows with each consumption

Two mailbox

Initially, the mayproduce mailbox is full of null messages

A Solution to the Readers/Writers Problem Using Message Passing

西南交通大学 Southwest Jiaotong University

- A **controller process** that has access to the shared data area equipped with **three mailboxes**
- Other processes (reader&writer)
  - send a request message to the controller to access the shared data with receiving an "OK" reply message,
  - indicate completion of access with sending "finished" message.

- Control Flow of Controller
  - If count > 0, then a writer has made a request and is being made to wait to clear all active readers. Therefore, only "finished" messages should be serviced.
  - If count = 0, then the only request outstanding is a write request. Allow the writer to proceed and wait for a "finished" message.
  - If count < 0, then no writer is waiting and there may or may not be readers active. Service all "finished" messages first to clear active readers. Then service write requests, and then read requests.

```
void reader(int i)
{
    message rmsg;
        while (true) {
                rmsg = i;
            send (readrequest, rmsg);
            receive (mbox[i], rmsg);
            READUNIT ();
            rmsg = i;
            send (finished, rmsg);
        }
}
void writer(int j)
{
    message rmsg;
    while (true){
        rmsg = j;
        send (writerequest, rmsg);
        receive (mbox[j], rmsg);
        WRITEUNIT ();
        rmsg = j;
        send (finished, rmsg);
    }
}
```

```
void controller()
{
    while (true)
    {
        if (count > 0) {
            if (!empty (finished)) {
                receive (finished, msg);
                count++;
            }
            else if (!empty (writerequest)) {
                receive (writerequest, msg);
                writer_id = msg.id;
                count = count - 100;
            }
            else if (!empty (readrequest)) {
                receive (readrequest, msg);
                count--;
                send (msg.id, "OK");
            }
        }
        if (count == 0) {
            send (writer_id, "OK");
            receive (finished, msg);
            count = 100;
        }
        while (count < 0) {
            receive (finished, msg);
            count++;
        }
    }
}
```

Writer before reader

Count is used to achieve mutual exclusion which is initialized to be greater than the no. of readers

Summary

- **Central themes of Modern Operating System are:**
  - Multiprogramming, multiprocessing, distributed processing
  - Fundamental to these themes is **concurrency**
    - ➤ issues of conflict resolution and cooperation arise
- **Interact of concurrency processes**
  - Unaware of each other, resource competition
  - Indirectly aware, share common resource, such as memory and file
  - Directly aware, communication with each other
- **Mutual Exclusion**
  - In a set of concurrent processes, **only one of process** can access a given resource **at any time**
- **Synchronization**
  - Coordinates the **execution order of multiple processes** to enforce dependencies. Synchronization focuses on **collaboration** rather than just resource exclusivity.
- **Semaphores**
  - Used for signals among processes for mutual exclusion and synchronization
- **Monitors**
  - A programming structure, only one process can enter the monitor by calling **the monitor's procedures**
  - The **condition variables** are used to achieve synchronization
- **Messages Passing**
  - Useful for the enforcement of mutual exclusion discipline

Producer & Consumer
Writer & Reader